

TP5 - Les bases de Docker – Synthèse

Exercice – Pile LEMP

1/

Commande: `docker network create --driver bridge --subnet 10.10.10.0/24 --gateway 10.10.10.1 lemp_network`

Crée un réseau **lemp_network** de type **bridge**, avec le **sous-réseau 10.10.10.0/24** et la **passerelle 10.10.10.1**, qui servira à connecter les conteneurs de la pile LEMP (Nginx, MariaDB, PHP).

2/

Commande: `mkdir -p ServeurWeb/html`

Crée le répertoire **html** dans ton dossier **ServeurWeb**.

Commande: `echo "<h1>Bienvenue sur mon serveur LEMP</h1>" > ServeurWeb/html/index.html`
`echo "<?php phpinfo(); ?>" > ServeurWeb/html/info.php`

Crée deux fichiers :

- **index.html** : affiche un message de bienvenue.
- **info.php** : exécute la fonction `phpinfo()` pour afficher la configuration de PHP.

3/

Commande: `docker run --rm nginx cat /etc/nginx/conf.d/default.conf > default.conf`

Récupère le fichier de configuration par défaut de **nginx** depuis un conteneur temporaire.

Ensuite, dans le fichier **default.conf** :

1. **Supprimez toutes les lignes faisant référence à Apache** (généralement des commentaires ou directives non pertinentes pour nginx).

Commande: `docker run --rm nginx cat /etc/nginx/conf.d/default.conf > default.conf`

2. **Modifiez la section PHP** pour que le transfert des requêtes se fasse via FastCGI vers le conteneur nommé **php**

Commande: `nano default.conf`

3. Vérifiez que le répertoire de travail **/var/www/html** correspond bien à celui qui sera utilisé dans le conteneur **php**.

4/

Commande: `docker run -d --name php --network lemp_network --ip 10.10.10.10 -v $(pwd)/html:/var/www/html php:8.2-fpm`

5/

Commande: `docker run -d --name www --network lemp_network --ip 10.10.10.20 -p 80:80 -v "$(pwd)/default.conf:/etc/nginx/conf.d/default.conf" -v "$(pwd)/html:/var/www/html" nginx`

Cette commande crée un conteneur **www** basé sur **nginx**, connecté au réseau **lemp_network** avec l'adresse IP **10.10.10.20**.

Le **port 80** du conteneur est exporté pour permettre l'accès depuis la VM.

Le conteneur utilise le fichier de configuration **default.conf** situé dans le répertoire courant et sert les fichiers du dossier **html**.

```
tpintegrationcontinue-clairecausse on 1 dev [?] is v1.0-SNAPSHOT via  took 2s
$ docker run -d --name www --network lemp_network --ip 10.10.10.20 -p 80:80 -v "$(pwd)/default.conf:/etc/nginx/conf.d/default.conf" -v "$(pwd)/html:/var/www/html" nginx
c288d1c1f2668c177aac0823a9de4361e73424e2da532320020e1d83134371f6
(base)
tpintegrationcontinue-clairecausse on 1 dev [?] is v1.0-SNAPSHOT via  
```

6/

Commande: `curl localhost/index.html`

→ Depuis la **VM**, affiche le contenu du fichier **index.html** : le message de bienvenue s'affiche correctement.

Commande: `curl localhost/info.php`

→ Depuis la **VM**, affiche la page générée par **PHP**, contenant les informations de configuration (résultat de `phpinfo()`).

Sur la **machine hôte**, ouvre un navigateur et entre l'adresse :

- `http://[IP_de_la_VM]/index.html` → affiche la page de bienvenue.

- `http://[IP_de_la_VM]/info.php` → affiche la page d'informations PHP.

```
(base)
tpintegrationcontinue-clairecausse on ? dev [?:] is v1.0-SNAPSHOT via
• ) docker ps

CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS
8d9c3e406f3f   nginx    "/docker-entrypoint..." 5 seconds ago  Up 5 seco
(base)
tpintegrationcontinue-clairecausse on ? dev [?:] is v1.0-SNAPSHOT via
• ) curl localhost/index.html

<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
(base)
tpintegrationcontinue-clairecausse on ? dev [?:] is v1.0-SNAPSHOT via
```

7/

Commande: `docker run -d --name bd --network lemp_network mariadb:11.4.3`

Crée un conteneur nommé **bd**, basé sur l'image **MariaDB 11.4.3**, et le connecte au réseau **lemp_network** pour qu'il puisse communiquer avec les autres services de la pile LEMP.

```

tpintegrationcontinue-clairecausse on ? dev [?+] is v1.0-SNAPSHOT via
) docker run -d --name bd --network lemp_network mariadb:11.4.3
Unable to find image 'mariadb:11.4.3' locally
11.4.3: Pulling from library/mariadb
f14242b632d8: Pull complete
d0a18beac8bf: Pull complete
aca8a8e17e98: Pull complete
f401964c0dd9: Pull complete
2af7e62043e0: Pull complete
429085fba01f: Pull complete
9ffe471e6e02: Pull complete
1f6304731171: Pull complete
Digest: sha256:e3432369d4d432ec2a3d777ff84ffca11ec8c2188cf1b6a0551a393ae5d833bb
Status: Downloaded newer image for mariadb:11.4.3
982d55b14b3af093cff97e9423b29ed0bf64ca470a37237ce151d82f7cd305f8
(base)
tpintegrationcontinue-clairecausse on ? dev [?+] is v1.0-SNAPSHOT via took

```

8/

Commande: `docker ps -a`

Permet de vérifier que le conteneur **bd** a bien été créé. Son **statut** est probablement `Exited (1)` car il s'est arrêté juste après son lancement.

```

tpintegrationcontinue-clairecausse on ? dev [?+] is v1.0-SNAPSHOT via took 8s
) docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
982d55b14b3a   mariadb:11.4.3 "/docker-entrypoint.s..." 4 minutes ago  Exited (1) 4 minutes ago          bd
8d9c3e406f3f   nginx         "/docker-entrypoint..." 5 minutes ago  Up 5 minutes   0.0.0.0:80->80/tcp, [::]:80->80/tcp  www

```

Ce **code d'erreur (1)** signifie que **MariaDB n'a pas pu démarrer**, généralement parce qu'**aucun mot de passe root n'a été défini**.

MariaDB exige la variable d'environnement `MYSQL_ROOT_PASSWORD` (ou équivalent) pour initialiser correctement la base de données.

10/

Commande: `docker logs bd`

Affiche les journaux du conteneur **bd**.

Les logs indiquent que le démarrage de **MariaDB** a échoué, avec un message du type :

```

"You need to specify one of MARIADB_ROOT_PASSWORD,
MARIADB_ROOT_PASSWORD_HASH,
MARIADB_ALLOW_EMPTY_ROOT_PASSWORD and
MARIADB_RANDOM_ROOT_PASSWORD"

```

Cela confirme que **le conteneur ne peut pas s'initialiser sans mot de passe root**, ce qui provoque son arrêt immédiat.

11/

Le message indique que le conteneur **bd** existe déjà (même s'il est arrêté), donc Docker ne peut pas en recréer un avec le même nom.

Commande: `docker rm -f bd`

→ Supprime l'ancien conteneur **bd**.

Commande: `docker run -d --name bd --network lemp_network -e MYSQL_ROOT_PASSWORD=mariadb mariadb:11.4.3`

→ Recrée le conteneur **MariaDB** avec le mot de passe root défini à **mariadb**. Cette fois, il démarre correctement et reste en **statut "Up"**.

12/

Commande: `docker ps`

→ Vérifie que le conteneur **bd** est bien en **statut "Up"**, indiquant que **MariaDB** fonctionne.

Commande: `docker exec -it bd mariadb -u root -pmariadb`

→ Ouvre un terminal MariaDB dans le conteneur avec l'utilisateur **root** et le mot de passe **mariadb**.

Commande dans MariaDB: `SHOW DATABASES;`

→ Affiche la liste des bases disponibles
: **information_schema, mysql, performance_schema** et **test**.

```

tpintegrationcontinue-clairecausse on  dev [?:] is  v1.0-SNAPSHOT via 
o > docker exec -it bd mariadb -u root -pmariadb
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 11.4.3-MariaDB-ubu2404 mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database                |
+-----+
| information_schema      |
| mysql                   |
| performance_schema      |
| sys                     |
+-----+
4 rows in set (0.003 sec)

```

13/

Commande dans le conteneur MariaDB:

```

CREATE DATABASE bd1;
USE bd1;
CREATE TABLE table1 (id INT PRIMARY KEY, nom VARCHAR(50));

```

→ Ces données sont **stockées à l'intérieur du conteneur** `bd`, dans le répertoire interne `/var/lib/mysql`.

C'est là que **MariaDB** conserve ses fichiers de base de données (données, index, logs, etc.).

```

4 rows in set (0.003 sec)

MariaDB [(none)]> CREATE DATABASE bd1;
Query OK, 1 row affected (0.004 sec)

MariaDB [(none)]> USE bd1;
Database changed
MariaDB [bd1]> CREATE TABLE table1 (id INT PRIMARY KEY, nom VARCHAR(50));
Query OK, 0 rows affected (0.010 sec)

MariaDB [bd1]> |

```

14/

Commande: `docker rm -f bd`

→ Supprime complètement le conteneur **bd** et toutes ses données internes.

Commande: `docker run -d --name bd --network lemp_network -e MYSQL_ROOT_PASSWORD=mariadb mariadb:11.4.3`

```
tp integration continue - claire
● > docker exec -it bd mariadb

+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
```

→ Relance un nouveau conteneur **bd** vierge.

Commande: `docker exec -it bd mariadb -u root -pmariadb -e "SHOW DATABASES;"`

→ Affiche la liste des bases : seules les bases par défaut (**information_schema**, **mysql**, **performance_schema**, **test**) sont présentes.

La base **bd1** et la table **table1** ont disparu, car elles étaient stockées à l'intérieur du conteneur supprimé.

15/

Solution 1 – Monter un volume local (bind mount)

Commande: `docker run -d --name bd --network lemp_network \`
`-e MYSQL_ROOT_PASSWORD=mariadb \`
`-v $(pwd)/data:/var/lib/mysql \`
`mariadb:11.4.3`

```

tpintegrationcontinue-clairecausse on 1 dev [?:] is v1.0-SNAPSHOT via
• ) docker run -d --name bd --network lemp_network \
  -e MYSQL_ROOT_PASSWORD=mariadb \
  -v "$(pwd)/data:/var/lib/mysql" \
  mariadb:11.4.3

60d8e02710885bf83cf4a1456ed3cad0a49332760d8b747a84ee5a748f98f1b6
(base)
tpintegrationcontinue-clairecausse on 1 dev [?:] is v1.0-SNAPSHOT via

```

→ Les données de MariaDB sont stockées dans le dossier `data` de ta machine.

Même si le conteneur est supprimé, les fichiers restent sur la VM, et un nouveau conteneur les réutilisera automatiquement.

Solution 2 – Utiliser un volume Docker nommé

Commande: `docker volume create mariadb_data`

```

docker run -d --name bd --network lemp_network \
-e MYSQL_ROOT_PASSWORD=mariadb \
-v mariadb_data:/var/lib/mysql \
mariadb:11.4.3

```

```

tpintegrationcontinue-clairecausse on 1 dev [?:] is v1.0-SNAPSHOT via
• ) docker volume create mariadb_data
docker run -d --name bd --network lemp_network \
  -e MYSQL_ROOT_PASSWORD=mariadb \
  -v mariadb_data:/var/lib/mysql \
  mariadb:11.4.3
  mariadb_data
  a4febdb049d843b16373767cb213c0186cbcb3f18d005ce5b1479d3f424de062
  (base)
tpintegrationcontinue-clairecausse on 1 dev [?:] is v1.0-SNAPSHOT via

```

→ Les données sont stockées dans un **volume géré par Docker** (visible avec `docker volume ls`).

Ce volume persiste indépendamment des conteneurs et peut être réutilisé par d'autres instances MariaDB.

16/

fichier **test_connexion.php** dans le répertoire **html** contenant le code suivant :

```

<?php
$servername = "bd";

```



```

$username = "root";
$password = "mariadb";
$dbname = "bd1";

$conn = new mysqli($servername, $username, $password, $dbname);

if ($conn->connect_error) {
    die("Échec de la connexion : " . $conn->connect_error);
}
echo "Connexion réussie à la base de données MariaDB !";
$conn->close();
?>

```

Ce script permet de tester la connexion entre les conteneurs **php** et **bd** sur le réseau **lemp_network**.

17/

Commande: `docker rm -f php`

On supprime l'ancien conteneur PHP.

Commande: `docker run -d --name php --network lemp_network --ip 10.10.10.10 -v $(pwd)/html:/var/www/html php:8.2-fpm`

On recrée le conteneur **php** sur le réseau **lemp_network**, avec l'adresse IP **10.10.10.10** et le répertoire **html** monté dans **/var/www/html**.

J'ai du utiliser Commande: `docker restart www`

18/

Commande: `docker exec -it php bash`

On ouvre un terminal dans le conteneur **php**.

Commande dans le conteneur: `docker-php-ext-install mysqli`

Il faut installer l'extension **MySQLi** nécessaire pour que PHP puisse communiquer avec MariaDB.

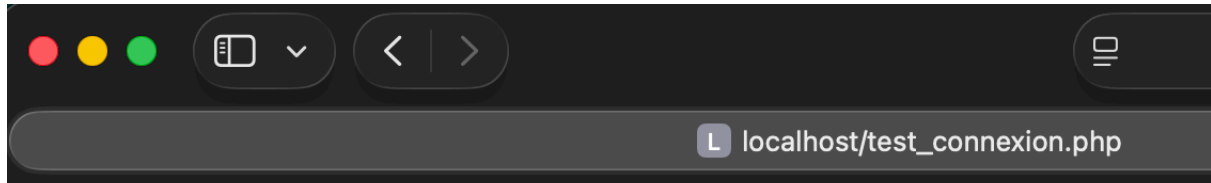
Commande: `docker restart php`

Ensuite, teste à nouveau ton script avec :

`php /var/www/html/test_connexion.php` ou dans le navigateur
via `http://localhost/test_connexion.php` .

```
tpintegrationcontinue-clairecausse on ? dev [?:] is 🐳 v1.0-SNAPSHOT via 🔌 took 8s
• 📄 docker exec -it php php /var/www/html/test_connexion.php

Connexion réussie à la base de données MariaDB !
What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → 🔌 docker debug php
  Learn more at https://docs.docker.com/go/debug-cli/
(base)
tpintegrationcontinue-clairecausse on ? dev [?:] is 🐳 v1.0-SNAPSHOT via 🔌
```



Connexion réussie à la base de données MariaDB !